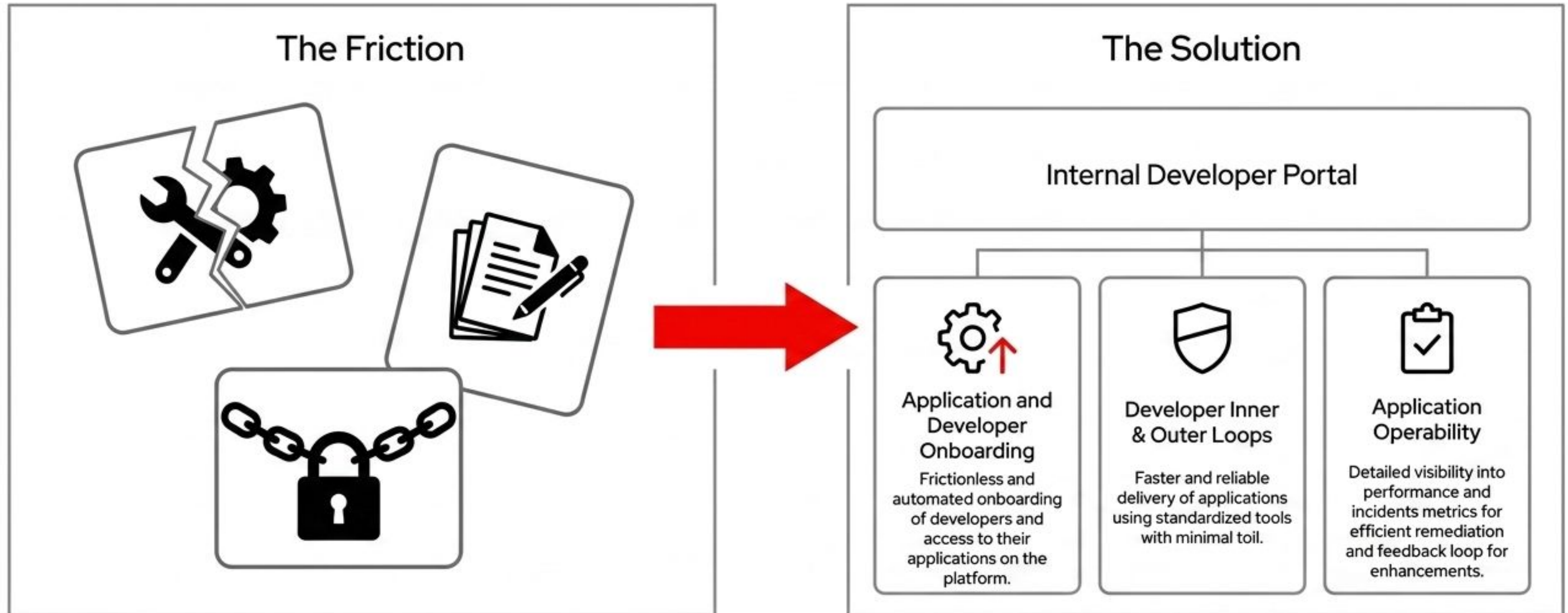


Red Hat Developer Hub Architecture

Alfred Bach

Principal Instructor

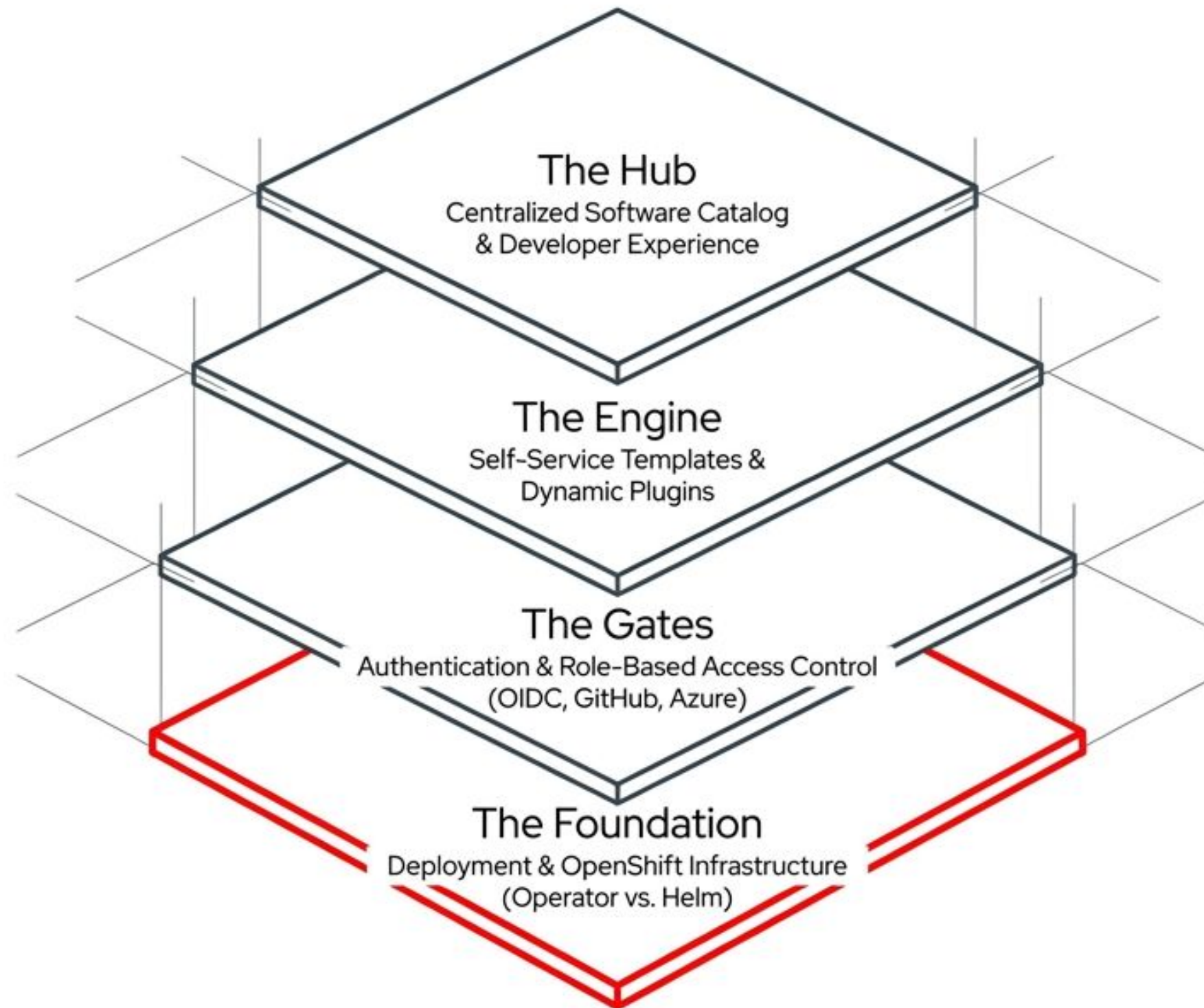
Scaling Cloud-Native Development Creates Operational Friction



Developers navigate fragmented toolchains, disparate documentation, and manual security gates, creating cognitive overload.

An Internal Developer Portal (IDP) acts as a centralized "one-stop shop" for documentation, collaboration, and self-service automation.

The Layered Architecture of Red Hat Developer Hub



Evaluating Deployment Strategies on OpenShift

	Operator-based Installation	Helm-based Installation
Configuration Mechanism	Uses the Backstage Custom Resource (CR) with spec.deployment.patch fields.	Uses custom values.yaml and app-config-rhdh ConfigMaps.
Configuration Injection	Both methods utilize OpenShift ConfigMaps for files and Secrets for environment variables.	
Root Filesystem	Patches securityContext.readOnlyRootFilesystem: true onto the backend container.	Configured via containerSecurityContext in values.yaml .
	✓ Fully supported for production environments.	✓ Fully supported for production environments.

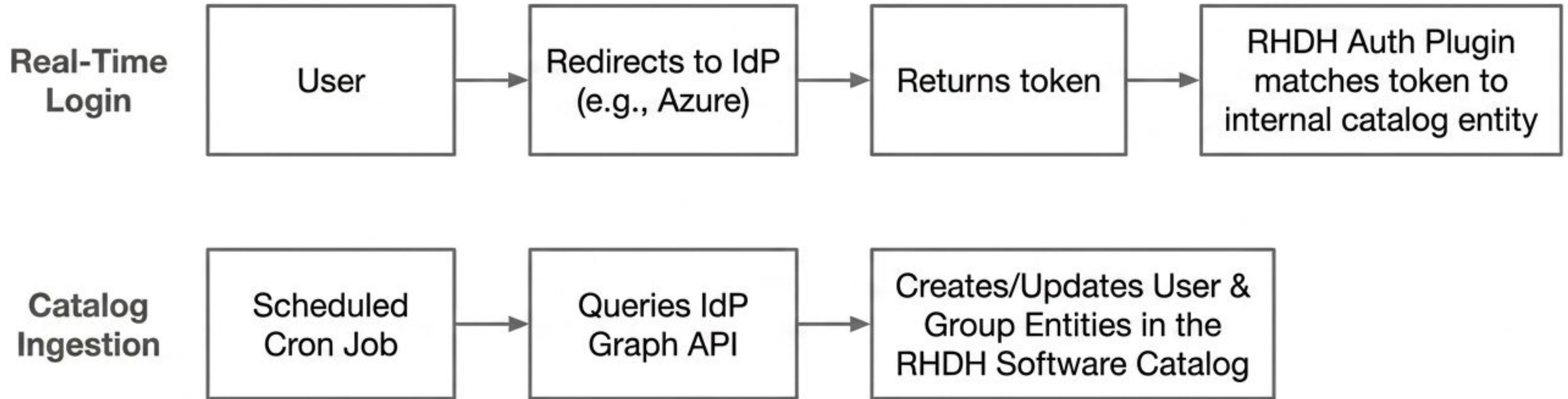
Provisioning Custom Configurations and Production Readiness



Securing Access with External Identity Providers

Red Hat Build of Keycloak (OIDC)	Microsoft Azure	GitHub
Resolver: oidcSubClaimMatchingKeycloakUserId Plugin: backstage-community-plugin-catalog-backend-module-keycloak-dynamic	Resolver: userIdMatchingUserEntityAnnotation Plugin: backstage-plugin-catalog-backend-module-msgraph-dynamic Detail: Requires Microsoft Graph API (User.Read.All, GroupMember.Read.All).	Resolver: usernameMatchingUserEntityName Plugin: backstage-plugin-catalog-backend-module-github-org-dynamic Detail: Uses GitHub Apps for fine-grained permissions and short-lived tokens.

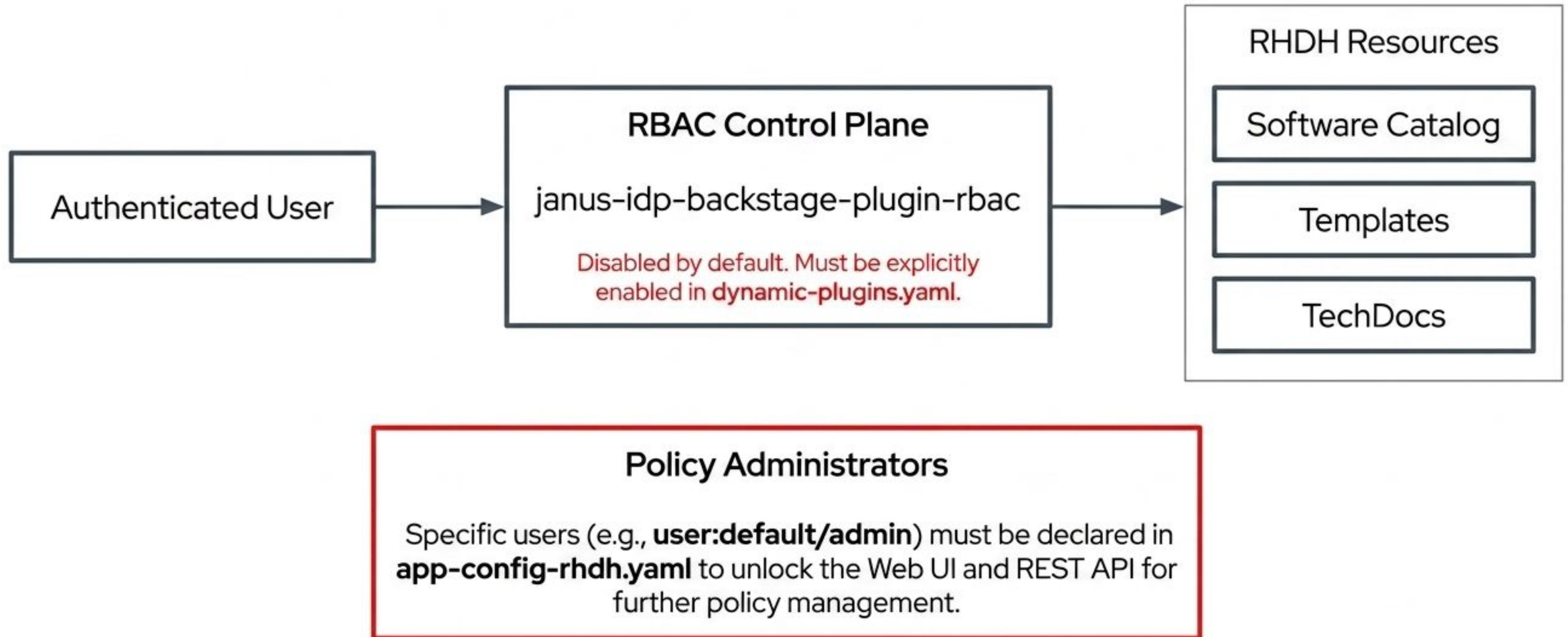
The Asynchronous One-Way Identity Synchronization Model



Manual Deletion is Ineffective.

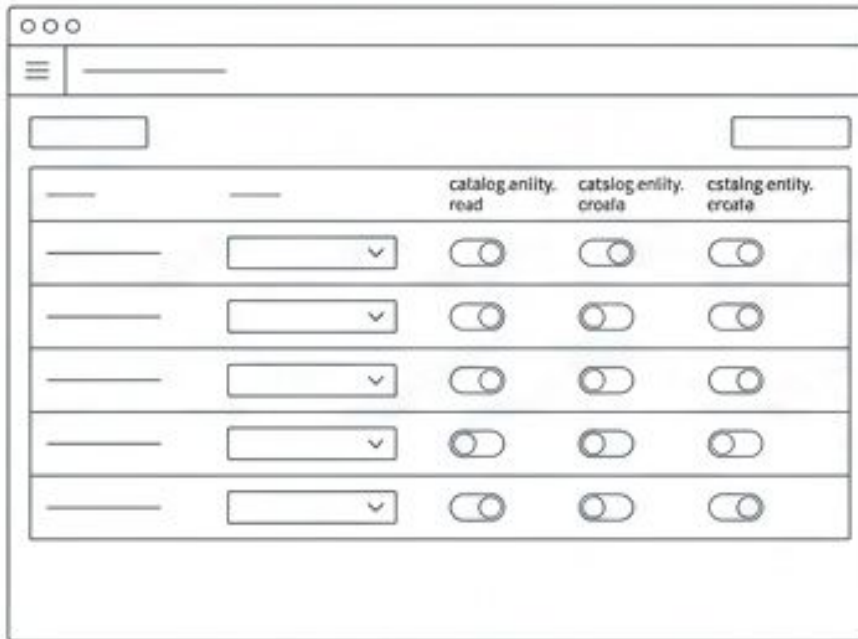
Because data flows one-way from the IdP to the catalog, deleting users manually via the RHDH UI or REST API will result in entities being recreated during the next ingestion cycle.

Governing Access with Role-Based Access Control (RBAC)



Three Avenues for RBAC Policy Configuration

The Web UI



The screenshot shows a web interface for managing RBAC policies. It features a table with columns for roles and permissions. The roles listed are 'catalog.entity.read', 'catalog.entity.create', and 'catalog.entity.create'. The permissions listed are 'read', 'create', and 'create'. The table has five rows, each representing a role and its associated permissions. The first row shows 'catalog.entity.read' with 'read' permission. The second row shows 'catalog.entity.create' with 'create' permission. The third row shows 'catalog.entity.create' with 'create' permission. The fourth row shows 'catalog.entity.create' with 'create' permission. The fifth row shows 'catalog.entity.create' with 'create' permission.

Policy administrators dynamically allocate roles, map users/groups, and define granular plugin permissions (`catalog.entity.read`, `catalog.entity.create`).

The REST API

```
Authorization: Bearer token
Content-Type: application/json

GET /api/permission/policies
Request token
Content-Type: json

{
  "_data": {
    "script": []
  },
  {
    "conditionas": },
    "roles": []
  }
```

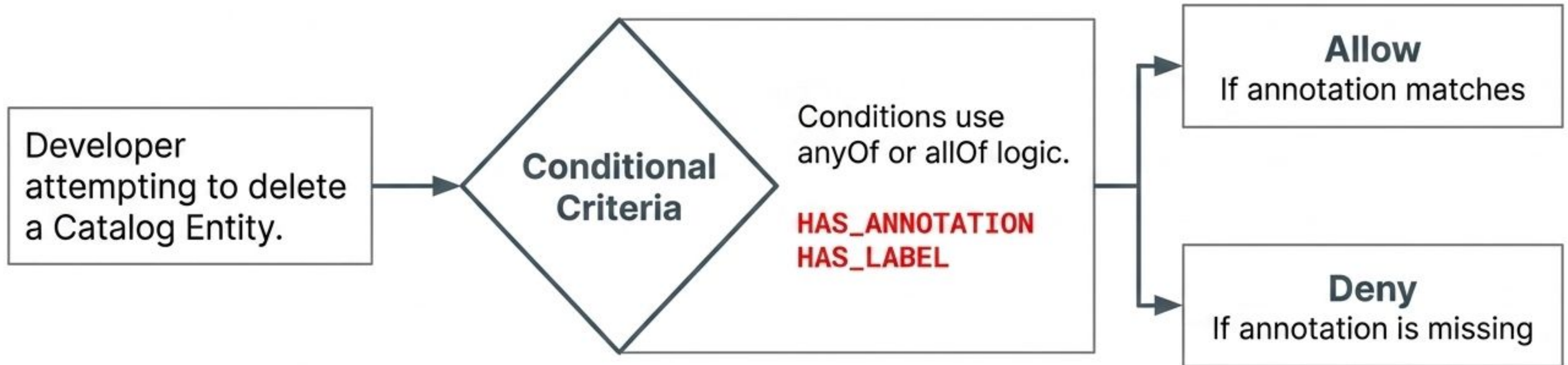
Enables automated maintenance and programmatic audits of roles and conditional rules.

External Configuration (GitOps)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: rbac-policy
data:
  rbac-policies.csv: |
    p, user:default/alice, role:admin,
    catalog.entity.read, allow
    g, group:default/developers, role:viewer,
    catalog.entity.read, allow
  rbac-conditional-policies.yaml: |
    - role: admin
      conditions: ...
```

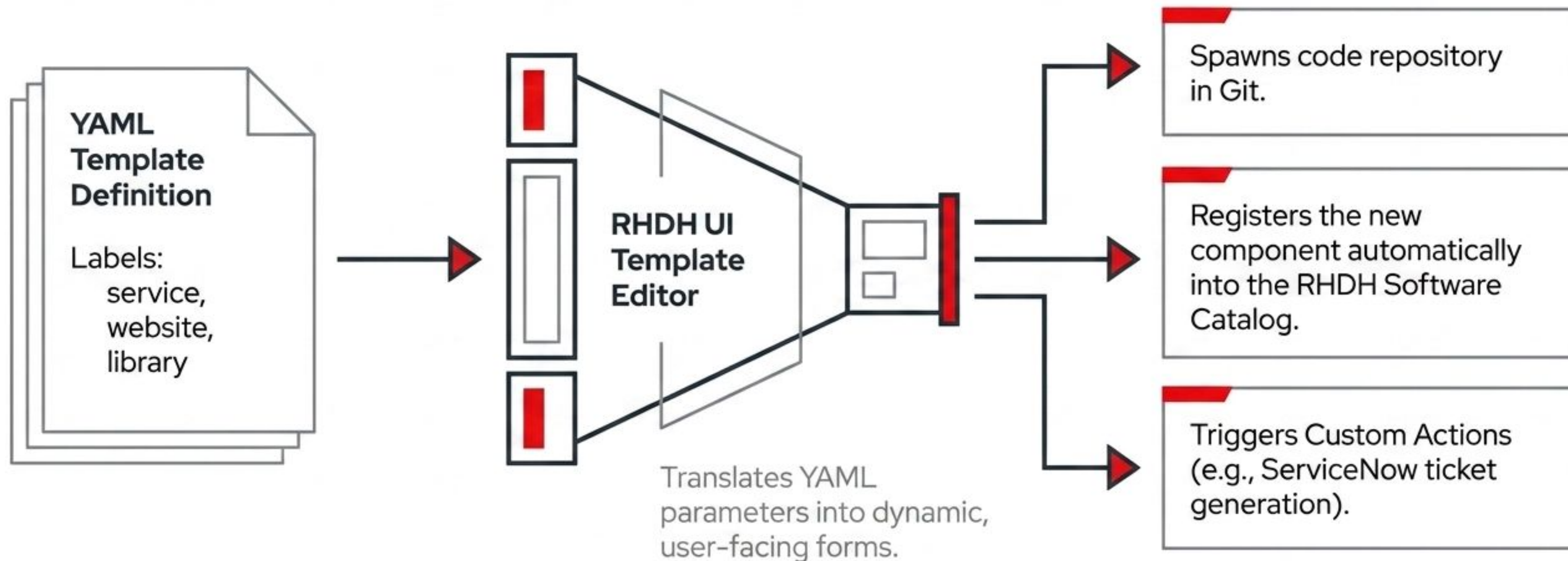
Defining `rbac-policies.csv` and `rbac-conditional-policies.yaml` externally, mounting them as ConfigMaps via the Operator or Helm to enforce immutable access control.

Dynamic Governance using Conditional Policies



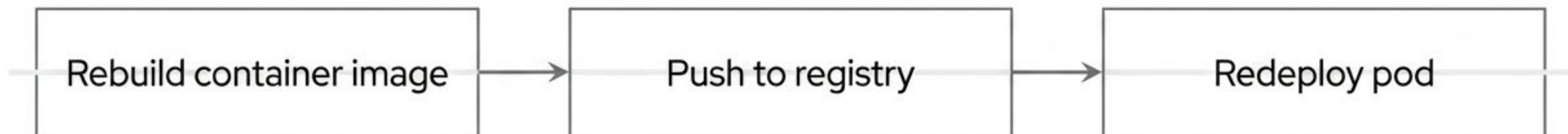
Conditions operate as content filters provided by the RBAC backend, evaluated in real-time against the specific catalog entity requested.

Orchestrating Self-Service via Software Templates



Zero-Rebuild Extensibility: Dynamic Plugin Architecture

Traditional Monolithic Approach



RHDH Dynamic Approach

Install external plugins without rebuilding the application container.
Changes apply upon pod restart.



OCI Images
(Support for `inherit` version tagging)

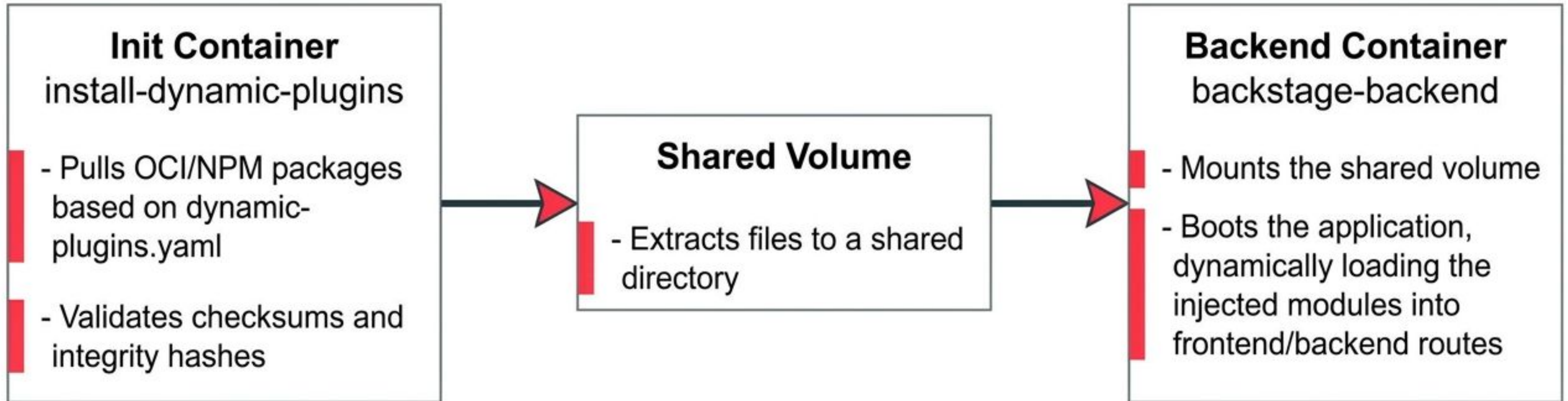


TGZ Archives
(Requires integrity hashing)



NPM Packages

The Dynamic Plugin Installation Pipeline



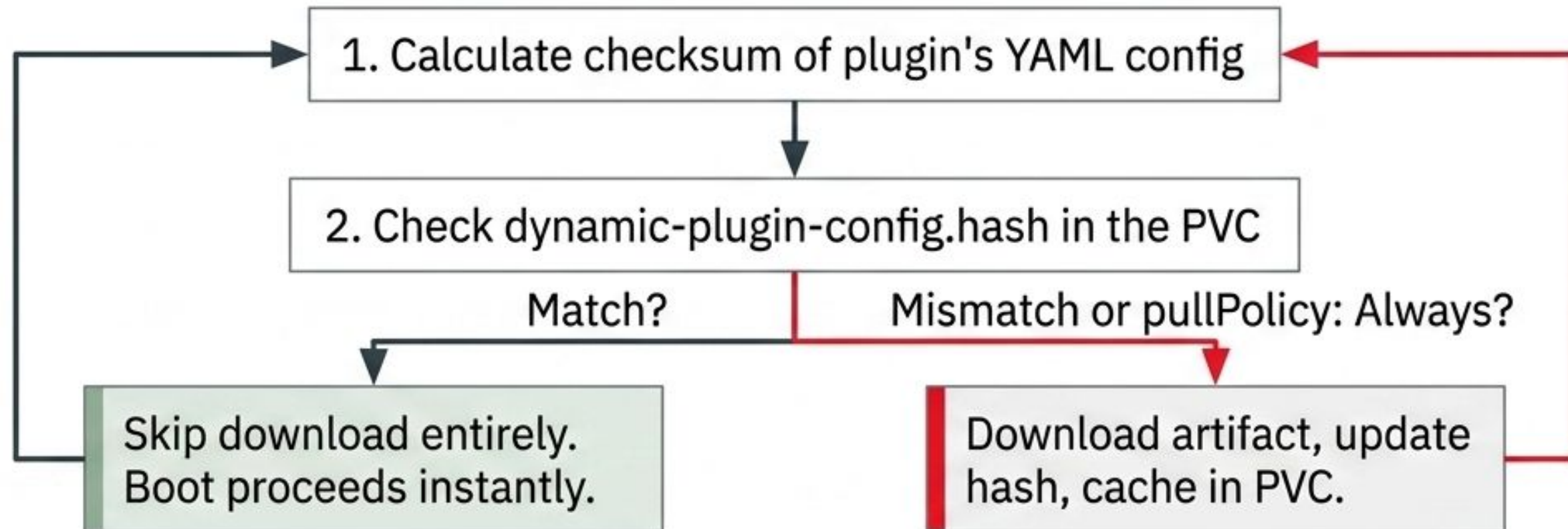
Optimizing Platform Boot Times with PVC Caching

The Problem

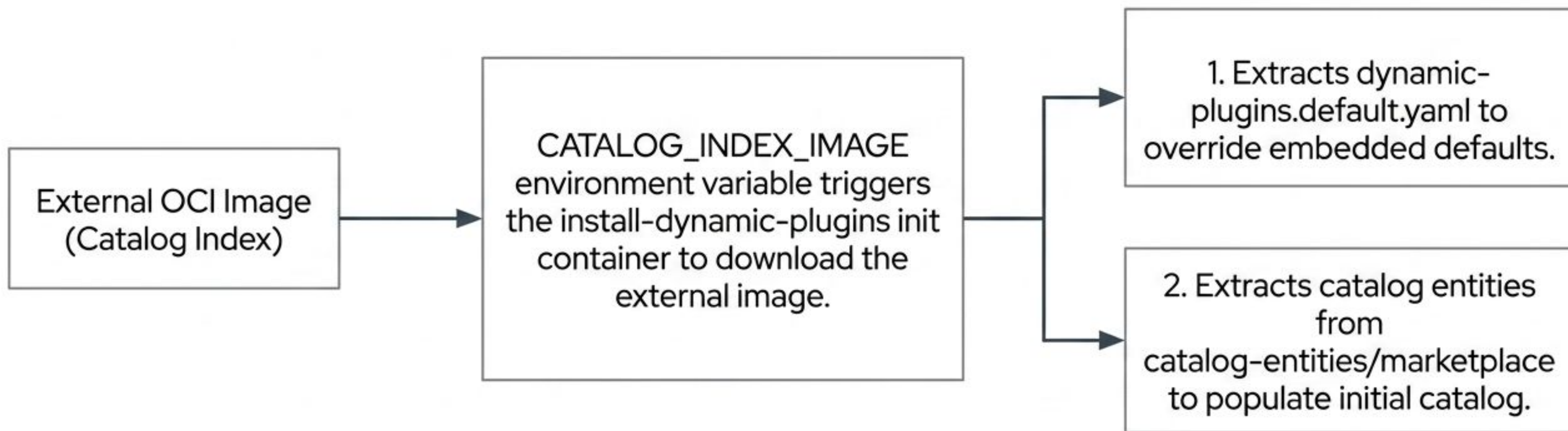
Downloading and extracting dozens of plugins on every pod restart severely impacts RHDH boot time.

The Solution

Mount dynamic-plugins-root as a Persistent Volume Claim (PVC).



Centralizing Configuration with Catalog Index Images



Insight: This pattern allows platform engineers to update plugin defaults across multiple RHDH instances without altering the core RHDH container image.

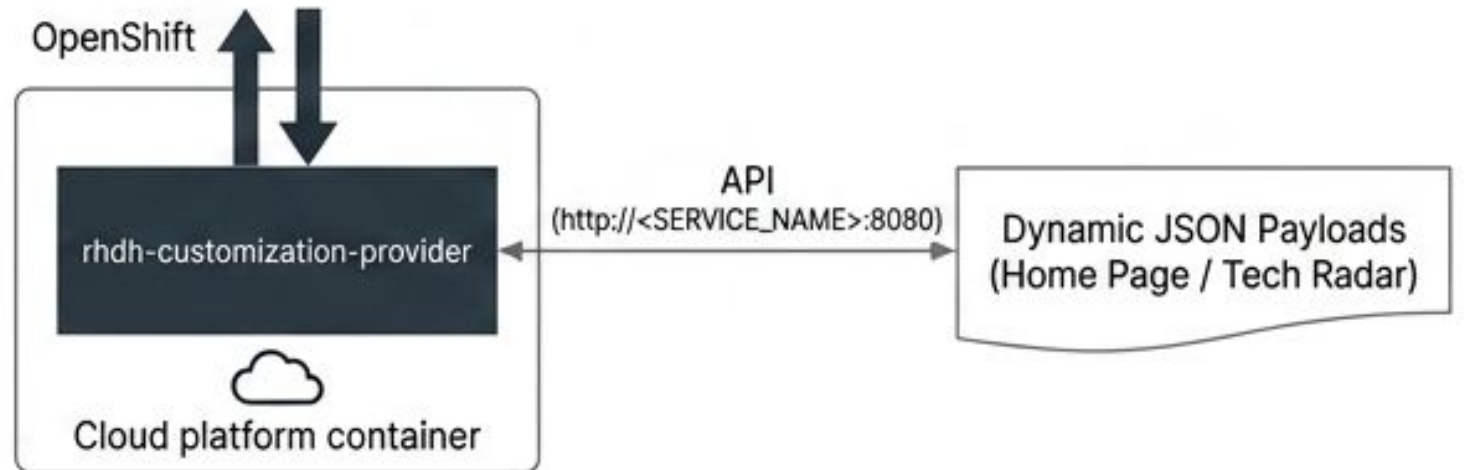
Customizing the Developer Interface

Method 1: Static Hosting



- JSON files hosted on GitHub/GitLab.
- app-config.yaml points directly to the raw URL.

Method 2: API Provider Service



- A dedicated microservice (e.g., rhdh-customization-provider) running on OpenShift.
- Exposes an API (`http://<SERVICE_NAME>:8080`) returning dynamically generated JSON payloads for the Home Page and Tech Radar proxies.

Platform Observability and Licensing Audits

Licensed User Statistics

userEntityRef displayName email lastTimeLogin				
_____	_____	_____	_____	
_____	_____	_____	_____	

Powered by the `licensed-users-info-backend` plugin.
Enables transparent tracking of active users against purchased entitlements.

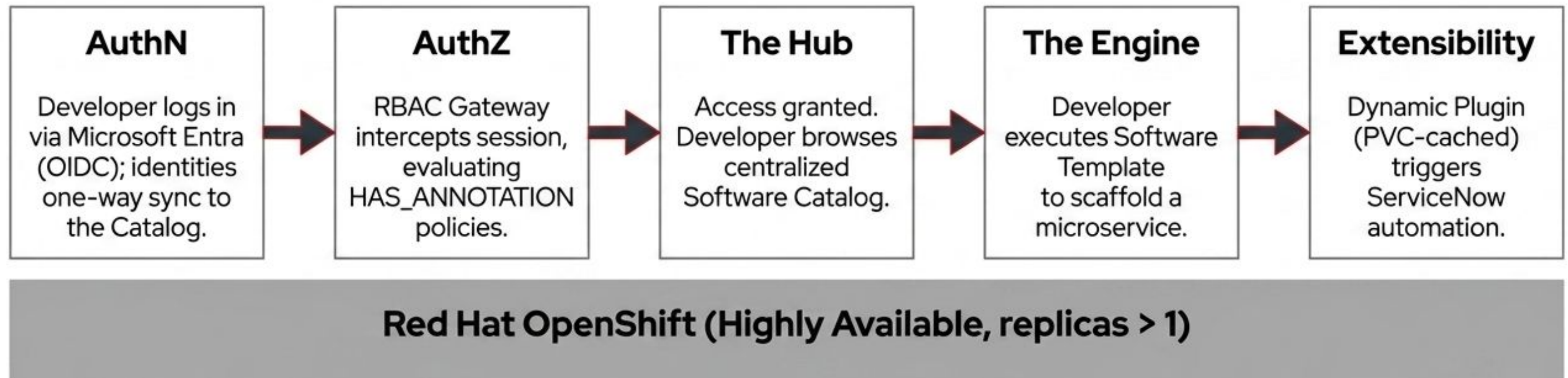
Logs & Events

ooo

```
[2024-05-20T10:01:15Z] INFO Starting pod initialization...
[2024-05-20T10:01:17Z] DEBUG Connection established to database...
[2024-05-20T10:01:19Z] WARN High memory usage detected...
[2024-05-20T10:01:21Z] ERROR Failed to process request: timeout...
```

Using the `kubernetes.proxy` permission.
Using the `kubernetes.proxy` permission. Allows users to a pot logs and events directly within the RHDH interface without requiring direct OpenShift console access.

Synthesis: The Orchestrated Developer Journey

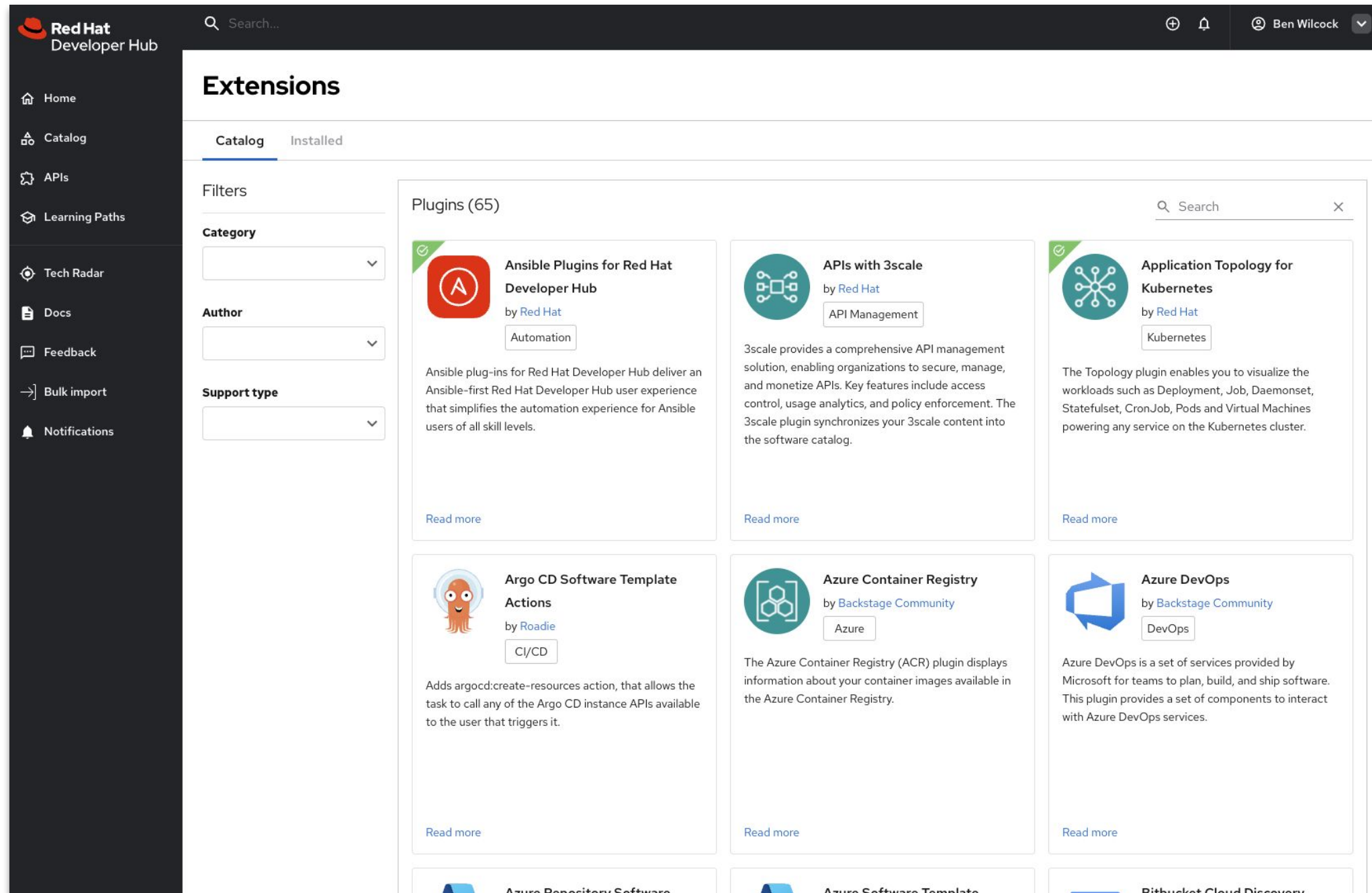


Frictionless Development, Engineered Governance

- ✓ **Deploy:** Declarative installation via OpenShift Operators with immutable configurations.
- ✓ **Secure:** Centralized OIDC identity paired with granular, condition-based RBAC policies.
- ✓ **Scale:** Zero-rebuild extensibility via PVC-cached dynamic plugins.

Simplified Extensibility for Developer Hub

Extensions Catalog for Plugins and Actions



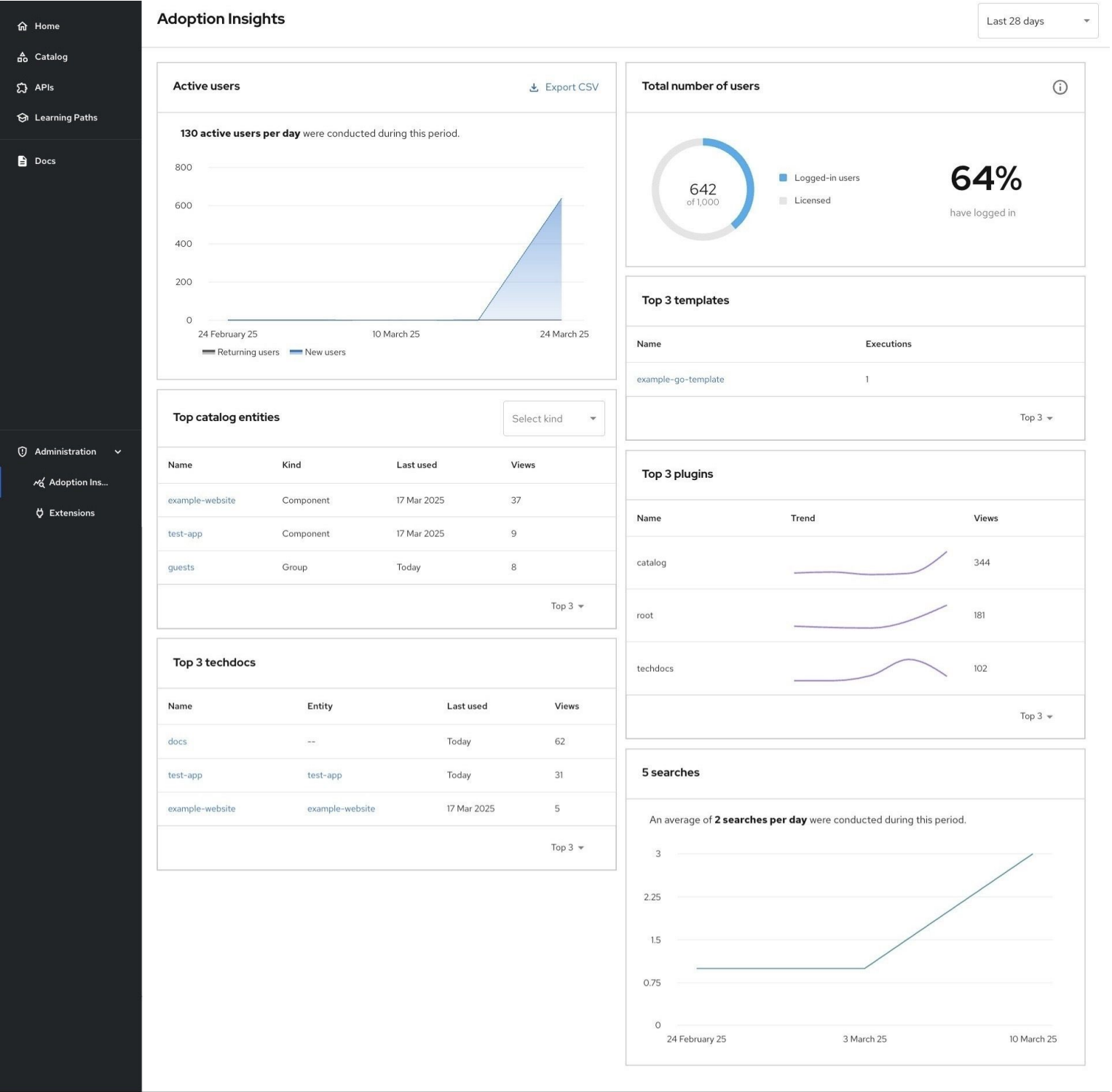
Key Features

The Extensions Catalog offers RHDH Admins information on the various plugins available on RHDH:

- **Long list of plugins:** Over 80 plugins are listed in the initial rollout with the potential for more to be added.
- **Detailed plugin information:** Includes descriptions and links to further information about each plugin.
- **Verified plugins:** A green badge allows Admins to see plugins included in their RHDH support subscription.
- **Certified plugins:** A purple badge allows Admins to see plugins supported by Red Hat's partners.
- **Custom plugins:** An orange badge shows plugins Admins have added from sources you trust.
- **Community plugins:** A broad selection of community plugins - popular with other users and the Backstage community.

Drive Platform Adoption

Visibility into User Activity, Trends, and Platform Usage



Key Features

The Adoption Insights plugin offers a range of features designed to help organizations monitor and improve their developer experience:

- **Active Users:** Track the number of active users over a specific time range, with breakdowns of new and returning users.
- **Total Users:** Compare logged-in users versus licensed users.
- **Top Catalog Entities:** Identify the most accessed components, APIs, and documentation entries.
- **Top Templates:** View the most commonly used templates in a ranked list.
- **Top TechDocs:** Analyze which documentation entries receive the most views.
- **Top Plugins:** Monitor plugin usage and popularity trends.
- **Portal Searches:** Visualize search trends and user queries within the portal.

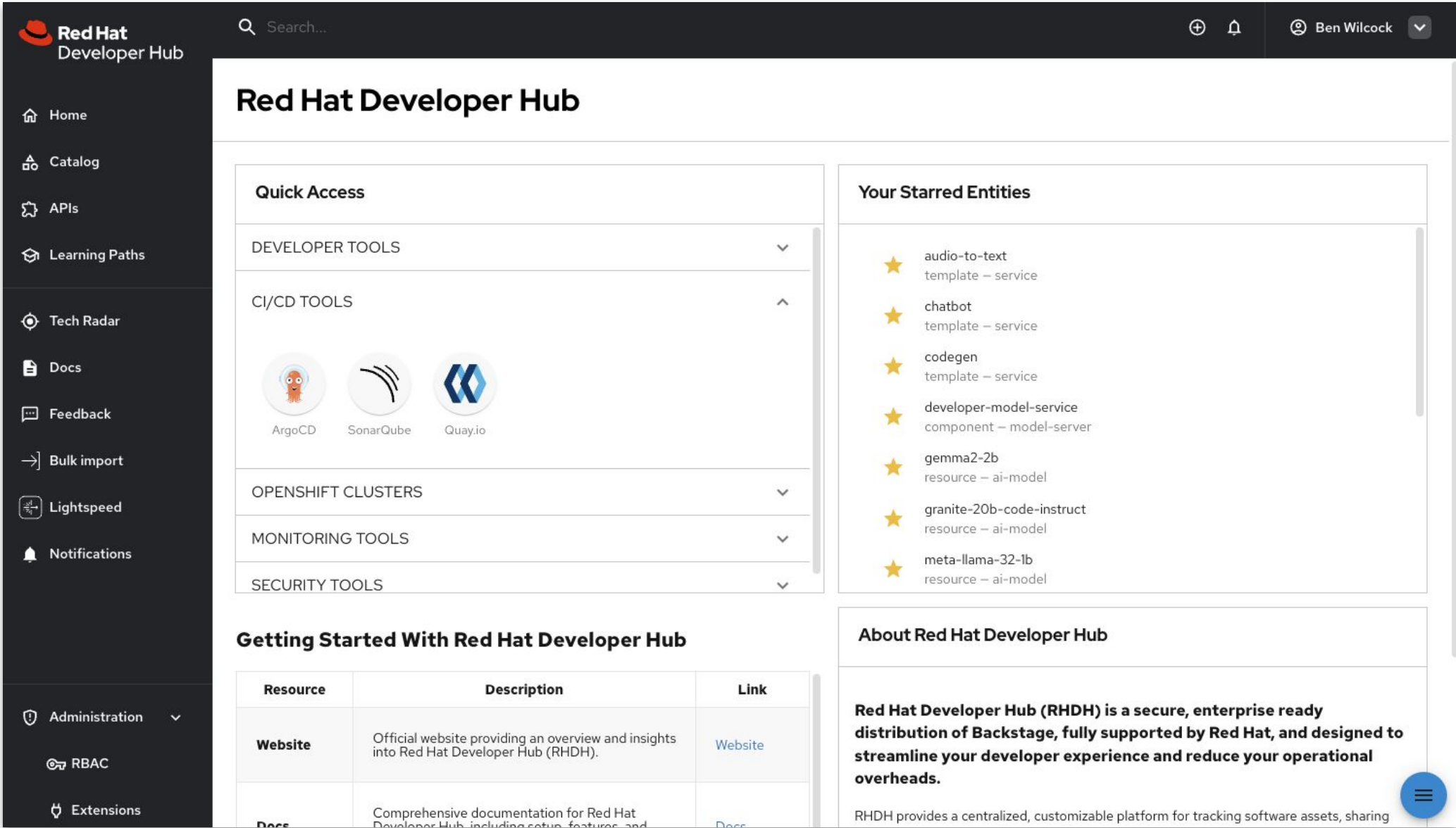
Build and Test Platform Enhancement Locally

RHDH Local for Plugin/Template Development

Key Features

RHDH-Local is perfect for busy Platform Engineers and Developers. With RHDH-Local you can:

- **Run RHDH Without Kubernetes:** RHDH Local removes the need to setup RHDH in an Kubernetes for development/testing –just install Docker or Podman, [download the code](#), and run locally for platform engineers
- **Experiment with Config, Content & Plugins:** Need to tweak a homepage layout, test a new dynamic plugin, or test configuration? RHDH Local provides a safe playground to validate these changes before rolling them out to a larger team.
- **Simple, Fast, and Disposable:** Because it runs in a containerized environment, you can spin up RHDH Local in minutes and tear it down just as easily when you're done—without cluttering your system.
- **Iterate Fast On Your IDP Content:** For developers working on custom TechDocs or Catalog items, RHDH Local offers a fast and simple solution that lets you test easily and have full control over the database.
- Supports x86_64 (amd64) and ARM



github.com/redhat-developer/rhdh-local



RHDH Orchestrator

Advanced Workflow Templates for Enterprises

Key Features

Allows for more sophisticated automation in software templates

Templates can make use of following features in the workflow definitions

- Eventing,
- Notifications,
- Approvals,
- Retry,
- Long running tasks

The screenshot displays the RHDH Orchestrator interface. The top navigation bar includes the Red Hat Developer Hub logo and a search bar. The main content area is divided into two sections: 'Details' and 'Results'. The 'Details' section shows the workflow 'create-ocp-project' with a status of 'Active' and a duration of '---'. The 'Results' section displays a JSON output for the workflow data. Below these sections is a 'Notifications' panel with filters and a list of notifications, including 'Project rhdh-test active' and 'Jira ticket created'.

Workflow Details:

CATEGORY	STATUS	DURATION
Infrastructure	Active	---

Workflow Results:

```
{ 1 item
  "workflowdata": { 7 items
    "response": [ 1 item
      0 : { 5 items
        "id" : string "7364c5b3-0443-4c9a-8924-86cf02ec167c"
        "user" : string "user:default/user1"
        "origin" : string "orchestrator"
        "created" : float 1733904628.342
        "payload" : { 5 items
          "link" : 
```

Notifications:

View	Filter	X
New only		
Created after		
Any time		
Sort by		
Newest on top		
Severity		
Low		

Project rhdh-test active
Project rhdh-test active
orchestrator - Create OCP Project workflow - a few seconds ago

Jira ticket created
Jira ticket created
orchestrator - Create OCP Project workflow - 2 minutes ago

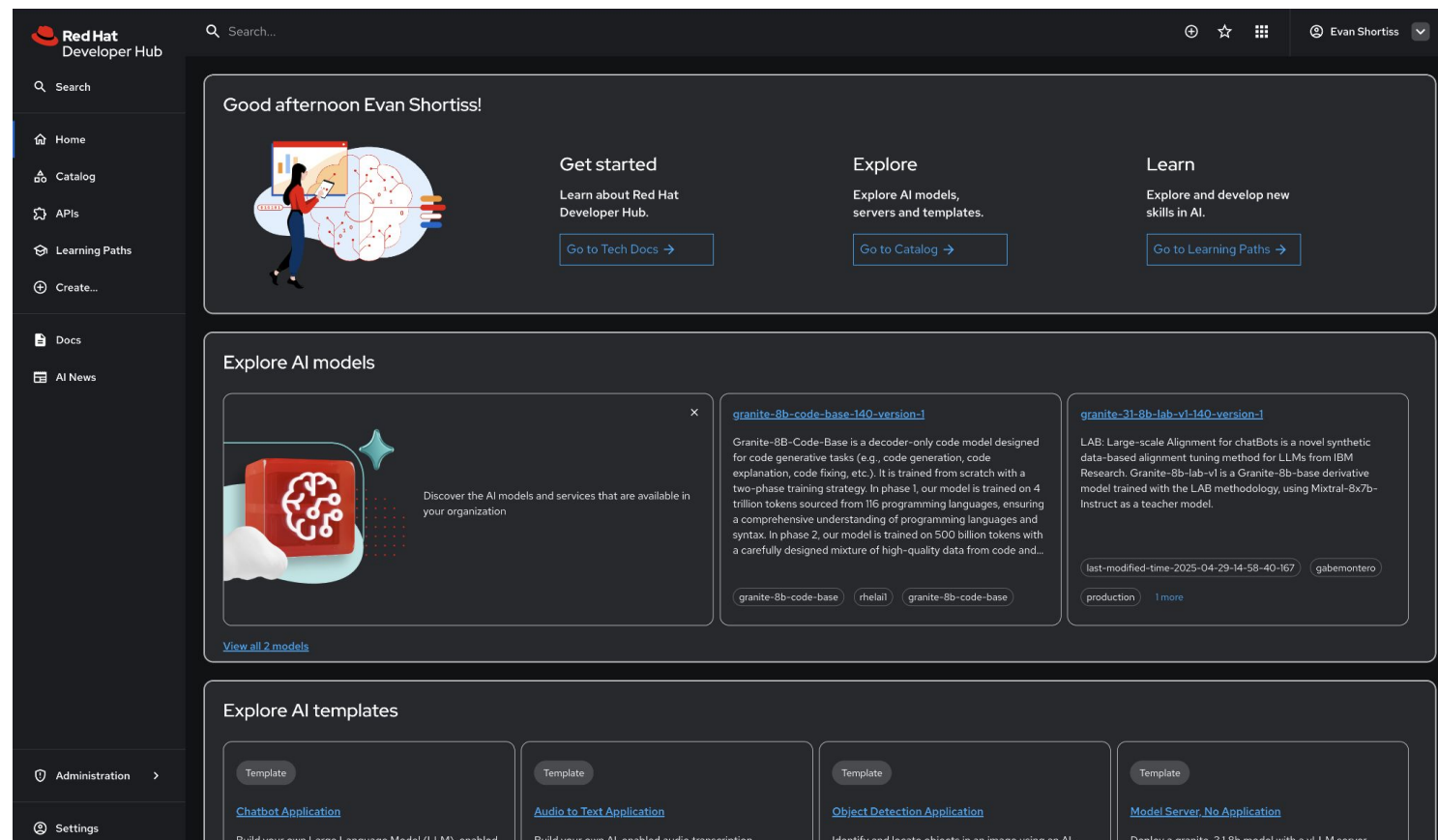
Developer Hub – Discover and Learn AI

Openshift AI Connector for Red Hat Developer Hub (Dev Preview)

Key Features

Provide a single, consistent hub for discovering, managing, and consuming AI models and model servers from Red Hat OpenShift AI .

- Automatically converts AI model artefacts into familiar catalog entities for developers.
- Imports model-server endpoints, URLs and authentication details into the software catalog.
- Syncs AI model metadata (descriptions, artifact URIs, authors/owners) seamlessly.
- Associates model cards and licensing links with familiar tech docs
- Provides a single developer portal view for AI models, endpoints, APIs and docs.



Developer Hub AI Templates

Enabling Quick Start for AI Applications with Software Templates

Red Hat Developer Hub

Search

Home

Catalog

APIs

Learning Paths

Create...

Docs

AI News

Administration

Settings

Search...

Software Templates

Create new software components using standard templates in your organization

Search

Personal

My Org

Categories

Tags

Owner

Templates

Service Audio to Text Application

Build your own AI-enabled audio transcription application. Pick from the...

ai whispercpp python asr

View TechDocs

Choose

Service Chatbot Application

Build your own Large Language Model (LLM)-enabled chat application. Pick from...

ai llamacpp vllm python

View TechDocs

Choose

Service Evan Test Chatbot Application

Build your own Large Language Model (LLM)-enabled chat application. Pick from...

ai llamacpp vllm python

View TechDocs

Choose

Service Code Generation Application

Build your own Large Language Model (LLM)-enabled code generation applicatio...

ai llamacpp vllm python

View TechDocs

Choose

Service Model Server, No Application

Deploy a granite-3.1 8b model with a vLLM server. While no application is configured,...

ai vllm modelserver

View TechDocs

Choose

Service Object Detection Application

Identify and locate objects in an image using an AI-enabled object detection application...

ai detr python

View TechDocs

Choose

Service RAG Chatbot Application

Enhance the capabilities of your Chatbot using a Retrieval-Augmented Generation...

ai llamacpp vllm python rag database

View TechDocs

Choose

25

 linkedin.com/company/red-hat

 youtube.com/@redhat

 facebook.com/RedHat

 x.com/RedHat